ROS 2 + Gazebo Robot Development

Pre-Build Laptop Workflow for a Student Robot Project

Purpose: Finish the robot's software architecture, simulation, interfaces, navigation logic and test plan on the laptop before physically building the robot.

Target: Ubuntu 24.04 + ROS 2 Jazzy + Gazebo + RViz2. Verify the exact Gazebo integration installed on your system rather than mixing commands from different Gazebo generations.

Project-specific note: The robot image/constraints folder is not attached or accessible in this chat right now. Therefore this first PDF is the master workflow template. After you upload the image and constraints, this can be made robot-specific.

Core rule: Do not start by buying motors. Convert the image and constraints into a specification, simulate the robot, test the ROS interfaces, and only then select hardware.

1. Complete Workflow

Robot image + constraints → requirements → ROS architecture → URDF/Xacro → Gazebo → drive/control → sensors → TF/odometry → RViz2 → SLAM → Nav2 → hardware selection → physical bring-up → validation.

Freebuff workflow

```
DISCOVER → PLAN → IMPLEMENT → BUILD → TEST → INSPECT → FIX → RETEST → VERIFY

Do not accept "code generated successfully".
Require:
- build passed
- simulator launched
- expected topics exist
- TF is correct
- behavior was actually tested
- acceptance criteria passed
```

Use one milestone at a time. Tell Freebuff to inspect the repository before editing, make the smallest required change, build, run the simulation/test, inspect errors, fix them, and repeat until the milestone acceptance tests pass.

2. Phase 0 — Convert the Robot Folder into a Specification

Create **PROJECT_REQUIREMENTS.md** from the image and constraint files. This is the source of truth for Freebuff.

Item	Information to determine
Robot type	Differential / Ackermann / mecanum / tracked / other
Dimensions	Length × width × height
Wheels	Count, diameter, arrangement
Wheel geometry	Wheelbase / track width
Mass & payload	kg / kg
Speed	Maximum required m/s
Turning	In-place / radius / steering angle
Surface	Indoor / outdoor / mixed
Sensors	LiDAR / camera / IMU / encoders / ultrasonic / etc.
Autonomy	Teleop / avoidance / SLAM / Nav2
Compute	Laptop / Pi / Jetson / other
Power	Battery voltage, capacity, runtime
Budget	Maximum allowed
Restrictions	Course, component, size, safety, cost constraints

- ■ Separate facts from assumptions.
- ■ Never invent dimensions from an image; mark unknowns as UNKNOWN.
- ■ Record every constraint that can invalidate hardware.
- ■ Define acceptance tests before implementation.

Example acceptance tests

```
AT-01 Robot spawns in Gazebo without errors.
AT-02 Dimensions and wheel placement match the specification.
AT-03 /cmd_vel moves the simulated robot.
AT-04 Joint states/odometry are published.
AT-05 TF tree is connected correctly.
AT-06 Required sensors publish realistic data.
AT-07 RViz2 displays robot and sensor data.
AT-08 SLAM works, if required.
AT-09 Nav2 reaches test goals, if required.
AT-10 Stop/safety behavior works.
```

3. Phase 1 — Laptop Preparation

Before building the robot package, verify the development machine.

- Ubuntu 24.04 works.
- ROS 2 Jazzy works in a fresh terminal.
- colcon works.
- RViz2 launches.
- Gazebo launches through the installed ROS-Gazebo integration.
- Git is installed.
- Freebuff can read/write the project and execute build/test commands.
- You can reproduce the environment from a new terminal.

Baseline

```
source /opt/ros/jazzy/setup.bash
ros2 doctor --report
ros2 pkg list | head
rviz2
```

Workspace convention

```
mkdir -p ~/sudeep_ws/src
cd ~/sudeep_ws
colcon build --symlink-install
source install/setup.bash
```

Keep your preferred rule of separate packages for separate experiments. For the final robot, however, use modular packages for description, bringup, control and navigation rather than one giant package.

4. Phase 2 — ROS 2 Robot Architecture

A typical mobile robot project can eventually look like:

```
~/sudeep_ws/src/
■■■ sudeep_robot_description/
■   ■■■ urdf/
■   ■■■ meshes/
■   ■■■ rviz/
■   ■■■ launch/
■■■ sudeep_robot_bringup/
■   ■■■ launch/
■   ■■■ config/
■■■ sudeep_robot_control/
■   ■■■ config/
■   ■■■ launch/
■■■ sudeep_robot_sensors/
■■■ sudeep_robot_navigation/
■   ■■■ config/
■   ■■■ maps/
■■■ sudeep_robot_interfaces/   # only if custom interfaces are needed
```

Core concepts

- ■ Nodes
- ■ Topics
- ■ Services
- ■ Actions
- ■ Parameters
- ■ TF2
- ■ URDF/Xacro
- ■ ros2_control
- ■ Gazebo
- ■ RViz2
- ■ Nav2

5. Phase 3 — Build the Digital Robot

The first real deliverable is a correct digital robot—not navigation code.

URDF/Xacro should contain

- ■ base_link
- ■ wheel links/joints
- ■ caster/additional wheels if present
- ■ sensor links
- ■ visual geometry
- ■ collision geometry
- ■ inertial properties
- ■ joint limits where appropriate
- ■ correct dimensions and wheel axes

Expected frame structure (adapt to the actual robot)

```
map
 ■■■ odom
      ■■■ base_link
           ■■■ left_wheel
           ■■■ right_wheel
           ■■■ lidar_link
           ■■■ imu_link
           ■■■ camera_link
```

Model validation

- ■ Correct scale
- ■ Wheels contact ground
- ■ No unexpected collision intersections
- ■ Robot does not fall through the floor
- ■ Plausible mass/inertia
- ■ Sensor locations match the intended physical mounting
- ■ Axes are correct

6. Phase 4 — Gazebo Simulation

Build the simulation in small steps.

Sequence

→ Spawn chassis only.

→ Add wheels and joints.

→ Add drive controller / ros2_control integration.

→ Test forward motion.

→ Test reverse.

→ Test rotation.

→ Test stopping.

→ Tune contact/friction assumptions.

→ Add sensors one at a time.

→ Create a repeatable launch sequence.

Drive test example

```
# Inspect actual topic/controller names first.
ros2 topic list

# Example only:
ros2 topic pub --rate 5 /cmd_vel geometry_msgs/msg/Twist \
"{linear: {x: 0.2}, angular: {z: 0.0}}"

# Rotation example:
ros2 topic pub --rate 5 /cmd_vel geometry_msgs/msg/Twist \
"{linear: {x: 0.0}, angular: {z: 0.5}}"
```

The exact controller and topic names depend on your configuration. Freebuff must inspect the actual files rather than assuming names.

7. Phase 5 — Sensors, Odometry and TF

Add and validate one sensor/subsystem at a time.

Sensor	Typical output	Verify
Encoders	joint states / odometry	direction, scale, update rate
LiDAR	/scan	range, frame, mounting
IMU	/imu	axes, frame, rate
Camera	image topics	resolution, frame, latency
Depth camera	depth topics	range, frame
Ultrasonic	range topics	limits, frame

Useful checks

```
ros2 topic list
ros2 topic echo /odom
ros2 topic echo /scan
ros2 topic hz /scan
ros2 topic info /cmd_vel
ros2 run tf2_tools view_frames
```

Odometry sanity checks

- ■ A commanded forward distance produces approximately the same simulated odometry distance.
- ■ Rotation changes yaw in the expected direction.
- ■ Left/right wheel directions are correct.
- ■ Repeated tests do not reveal an obvious scale/sign error.
- ■ TF sensor frames are attached to the correct parent.

8. Phase 6 — RViz2

Use RViz2 as the visual debugging dashboard.

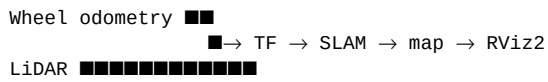
- RobotModel
- TF
- LaserScan if LiDAR exists
- Camera/image display if required
- Odometry
- Map if SLAM is used
- Path/planned trajectory if Nav2 is used

RViz acceptance

- Correct robot scale
- Connected TF
- LiDAR overlays the environment correctly
- Odometry direction is correct
- Map and robot pose align
- Navigation path uses the correct frame

9. Phase 7 — SLAM (Only If Required)

Do not start SLAM until drive, odometry, TF and sensors are stable.



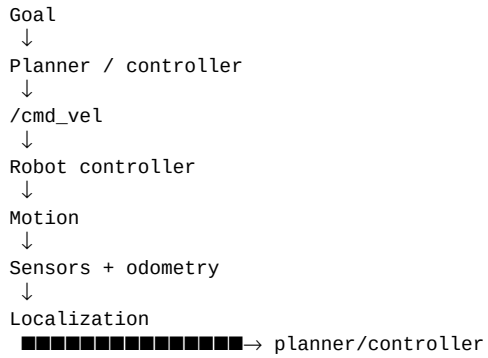
SLAM test sequence

1. Start simulator.
2. Start control.
3. Start LiDAR.
4. Verify /scan.
5. Verify odometry.
6. Verify TF.
7. Start SLAM.
8. Drive a simple loop.
9. Inspect map.
10. Save the map only after it is stable.

If the map rotates, stretches, doubles or drifts badly, first investigate sensor frames, odometry scale, timestamps, update rates and TF. Do not blindly compensate with navigation parameters.

10. Phase 8 — Nav2 (Only If Required)

Nav2 comes after reliable driving and localization.



- ■ Load a map.
- ■ Configure localization.
- ■ Configure costmaps.
- ■ Set robot footprint from actual dimensions.
- ■ Configure planner/controller.
- ■ Set velocity/acceleration limits from the requirements.
- ■ Test nearby goal.
- ■ Test longer goal.
- ■ Test obstacle behavior.
- ■ Test stopping/recovery.

Do not copy Nav2 parameters from another robot without checking footprint, speed, acceleration and sensor characteristics.

11. Phase 9 — Hardware Selection

Select hardware only after the simulated requirements are understood.

Motor sizing concept

```
Required wheel force ≈ rolling resistance
                     + slope force
                     + acceleration force

Wheel torque ≈ wheel force × wheel radius

Also check:
- gearbox ratio
- efficiency
- continuous/peak current
- motor-driver limits
- battery voltage
- duty cycle
```

Hardware gate

- ■ Motor voltage matches system.
- ■ Continuous and peak current fit driver limits.
- ■ Torque includes payload and slope.
- ■ Encoder resolution is adequate.
- ■ Wheel diameter matches simulation.
- ■ Driver supports intended interface.
- ■ Compute has enough resources.
- ■ Battery supports peak motor current.
- ■ Regulators support compute/sensor loads.
- ■ Emergency stop/safe power cut is planned.

12. Phase 10 — Physical Bring-Up

Never connect and enable the whole robot at once.

→ Verify power rails with motors disabled.

→ Boot robot computer.

→ Test one motor-driver channel.

→ Test one motor at low speed.

→ Test encoder feedback.

→ Test both wheels.

→ Verify odometry.

→ Mount/test IMU.

→ Mount/test LiDAR.

→ Test camera.

→ Verify real sensor TF positions.

→ Run low-speed teleoperation.

→ Test stop behavior.

→ Only then run autonomous navigation.

13. Freebuff Project Instruction

Place a version of the following in the project instructions. Replace placeholders after the robot image and constraints are supplied.

```
You are implementing a ROS 2 Jazzy robot simulation project.

SOURCE OF TRUTH
- Read PROJECT_REQUIREMENTS.md before changing code.
- Inspect the repository before editing.
- Never invent dimensions, sensors, topics, frames or hardware.
- Mark unknowns as assumptions and keep them configurable.

WORKFLOW
1. DISCOVER: inspect packages, launch files, URDF/Xacro, configs and tests.
2. PLAN: state the smallest changes for this milestone.
3. IMPLEMENT: make only those changes.
4. BUILD: run colcon build --symlink-install.
5. TEST: launch the relevant Gazebo simulation.
6. INSPECT: check topics, nodes, TF, logs and behavior.
7. FIX: diagnose the actual error.
8. RETEST: repeat until acceptance criteria pass.
9. VERIFY: run the relevant full test sequence.
10. REPORT: changed files, commands, tests, assumptions and next step.

DO NOT STOP AT "code written" OR "build passed".
A milestone is complete only when its behavior has been tested.

SAFETY
- Keep real-motor commands disabled during simulation development.
- Start physical tests at very low speed.
- Simulation success does not guarantee physical safety.
```

14. Project Milestones

M0	Requirements	Image/constraints → PROJECT_REQUIREMENTS.md
M1	Laptop	ROS 2 + Gazebo + RViz2 baseline
M2	Description	URDF/Xacro validated
M3	Simulation	Robot spawns correctly
M4	Drive	cmd_vel moves simulated robot
M5	Feedback	joint states + odometry
M6	TF	frame tree correct
M7	Sensors	required sensors publish
M8	RViz	debug display saved
M9	SLAM	mapping works if required
M10	Nav2	goal navigation if required
M11	Hardware	components selected
M12	Bring-up	subsystems tested individually
M13	Validation	physical acceptance tests pass

15. Final Pre-Build Gate

Do not begin the physical build until these are checked.

- ■ Robot dimensions known or explicitly assumed.
- ■ Wheel configuration known.
- ■ Mass/payload known.
- ■ Maximum speed defined.
- ■ Required sensors defined.
- ■ Compute selected.
- ■ URDF/Xacro works.
- ■ Gazebo model works.
- ■ Drive simulation works.
- ■ Odometry works.
- ■ TF tree correct.
- ■ Sensor topics work.
- ■ RViz2 works.
- ■ SLAM works if required.
- ■ Nav2 works if required.
- ■ Motor torque/current requirements calculated.
- ■ Motor driver compatible.
- ■ Battery/regulators sized.
- ■ Emergency stop planned.
- ■ Freebuff has reproducible build/test instructions.
- ■ A known-good simulation checkpoint is committed.

Checkpoint

```
git status
git add .
git commit -m "Known-good Gazebo simulation baseline"
```

Finished laptop stage

```
Launch sequence
    ↓
Gazebo
    ↓
Robot spawns
    ↓
Controllers
    ↓
Sensors
    ↓
TF + odometry
    ↓
RViz2
    ↓
SLAM / Nav2 (if required)
    ↓
Acceptance tests pass
    ↓
ONLY THEN → physical hardware
```

16. Files to Give Freebuff

Before asking Freebuff to implement the final robot, put these in the project folder:

- Robot reference image(s)
- Constraint/specification document
- PROJECT_REQUIREMENTS.md
- Ubuntu + ROS 2 version
- Existing workspace/package tree
- Course rubric/marking scheme
- Budget/component restrictions
- Required demonstration tasks
- Known hardware
- Acceptance-test checklist

Next step: Upload the robot image and the constraints folder/file here. I can then make a second, robot-specific PDF with the exact dimensions/assumptions, package structure, Gazebo model plan, sensor list, TF tree, commands, Freebuff master prompt, milestones and hardware-selection checklist.